

TP8 - GitOps : Git comme source de verite pour vos deploiements

Objectifs

A la fin de ce TP, vous serez capables de :

- Comprendre les principes fondamentaux du GitOps
- Utiliser Git comme source unique de verite pour la configuration des environnements
- Creer un pipeline CI/CD avec promotion d'environnements (staging -> production)
- Mettre en place un mecanisme de rollback via l'historique Git
- Suivre et verifier les deploiements avec un manifeste

Pre-requis

- Avoir complete les TP1 a TP7
- Avoir un compte GitLab fonctionnel
- Connaitre les bases de Git (commit, push, branches)
- Connaitre les bases d'un pipeline `.gitlab-ci.yml`

Etape 1 : Qu'est-ce que le GitOps ? (Theorie)

L'analogie simple

****Git est la telecommande de votre infrastructure.****

Imaginez votre infrastructure (serveurs, applications, configurations) comme une television. Sans GitOps, vous devez vous lever et appuyer sur les boutons de la television a chaque fois (actions manuelles sur les serveurs). Avec GitOps, vous avez une telecommande (Git) : vous appuyez sur un bouton (un commit), et la television change de chaine (le deploiement se fait automatiquement).

Les 3 piliers du GitOps

1. Git comme source unique de verite (Single Source of Truth)

Pourquoi ? Parce que tout le monde regarde au meme endroit.

Sans GitOps :

- "Quelle version est en production ?" -> "Je ne sais pas, il faut demander a Pierre"
- "Qui a change la config du serveur ?" -> "Aucune idee"

Avec GitOps :

- "Quelle version est en production ?" -> On regarde le depot Git

- "Qui a change la config ?" -> On regarde l'historique Git (git log)

Tout est dans Git : le code, la configuration, les variables d'environnement, les scripts de déploiement.

2. Declaratif vs Imperatif

Approche	Exemple	GitOps ?
Imperatif	"Connecte-toi au serveur, installe Node.js, copie les fichiers, redemarre le service"	Non
Declaratif	"L'application doit tourner en Node.js 18, sur le port 3000, avec 2 replicas"	Oui

L'approche declarative, c'est comme commander au restaurant : vous dites ce que vous voulez ("une pizza margherita"), pas comment le faire ("prenez de la farine, pétrissez la pâte...").

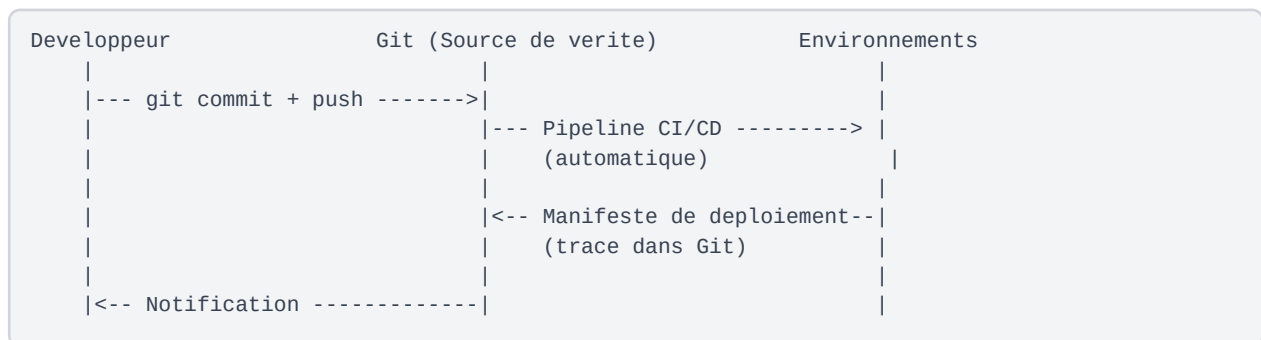
En GitOps, on décrit **l'état désiré** dans des fichiers de configuration, et le système se charge de rendre cet état réel.

3. Modele Pull vs Push

Modele	Comment ça marche	Exemple
Push	Le pipeline CI/CD envoie les changements vers le serveur	ssh serveur && deploy
Pull	Le serveur surveille Git et tire les changements	ArgoCD, Flux

Dans ce TP, nous utilisons un **modele Push simplifié** (le pipeline déploie), mais nous gardons les principes GitOps : tout est dans Git, tout est déclaratif, tout est traçable.

Schema recapitulatif



Etape 2 : Créer le projet sur GitLab

2.1 Créer un nouveau projet

1. Connectez-vous à GitLab
2. Cliquez sur **"New project"** (ou "Nouveau projet")
3. Choisissez **"Create blank project"**
4. Remplissez les informations :
 - **Nom du projet** : tp8-gitops

- **Visibilite** : Private
 - **Initialiser avec un README** : Oui
5. Cliquez sur **"Create project"**

2.2 Cloner le projet en local

```
git clone https://gitlab.com/VOTRE-USERNAME/tp8-gitops.git
cd tp8-gitops
```

Etape 3 : Preparer le code

3.1 Initialiser le projet Node.js

```
npm init -y
```

3.2 Installer les dependances

```
npm install express
npm install --save-dev jest
```

3.3 Creer le fichier principal `app.js`

```

const express = require('express');
const app = express();

// Lire la configuration depuis les variables d'environnement
const PORT = process.env.APP_PORT || 3000;
const NODE_ENV = process.env.NODE_ENV || 'development';
const LOG_LEVEL = process.env.LOG_LEVEL || 'info';

app.get('/', (req, res) => {
  res.json({
    message: 'Hello GitOps!',
    environment: NODE_ENV,
    version: process.env.APP_VERSION || '1.0.0',
    timestamp: new Date().toISOString()
  });
});

app.get('/health', (req, res) => {
  res.json({
    status: 'healthy',
    environment: NODE_ENV,
    uptime: process.uptime()
  });
});

// Export pour les tests
module.exports = app;

// Demarrer le serveur seulement si on execute directement le fichier
if (require.main === module) {
  app.listen(PORT, () => {
    console.log(`Serveur démarre sur le port ${PORT} en mode ${NODE_ENV}`);
    console.log(`Niveau de log: ${LOG_LEVEL}`);
  });
}

```

3.4 Créer un test simple `app.test.js`

```

const request = require('supertest');
const app = require('./app');

// Installer supertest : npm install --save-dev supertest

describe('GET /', () => {
  it('devrait retourner un message de bienvenue', async () => {
    const res = await request(app).get('/');
    expect(res.statusCode).toBe(200);
    expect(res.body.message).toBe('Hello GitOps!');
  });
});

describe('GET /health', () => {
  it('devrait retourner le statut healthy', async () => {
    const res = await request(app).get('/health');
    expect(res.statusCode).toBe(200);
    expect(res.body.status).toBe('healthy');
  });
});

```

3.5 Mettre à jour le `package.json`

Assurez-vous que la section scripts contient :

```
{
  "scripts": {
    "start": "node app.js",
    "test": "jest"
  }
}
```

Etape 4 : Créer les fichiers de configuration par environnement

Pourquoi mettre la configuration dans Git ?

C'est un principe cle du GitOps. Voici les raisons :

Avantage	Explication
Tracabilite	On sait qui a change quoi et quand (git log, git blame)
Audit	On peut prouver l'etat de la configuration a n'importe quel moment
Rollback	Si une config casse tout, on revient en arriere avec Git
Review	Les changements de config passent par des Merge Requests
Reproductibilite	N'importe qui peut recreer l'environnement a partir de Git

4.1 Créer le dossier `environments/`

```
mkdir -p environments
```

4.2 Créer le fichier `environments/staging.env`

```
APP_PORT=3001
NODE_ENV=staging
LOG_LEVEL=debug
API_URL=https://staging-api.example.com
```

Pourquoi ces valeurs ?

- `APP_PORT=3001` : Port different de la production pour eviter les conflits
- `NODE_ENV=staging` : Permet au code de savoir qu'il est en staging
- `LOG_LEVEL=debug` : En staging, on veut TOUS les logs pour deboguer
- `API_URL` : Pointe vers l'API de staging, pas celle de production

4.3 Créer le fichier `environments/production.env`

```
APP_PORT=3000
NODE_ENV=production
LOG_LEVEL=warn
API_URL=https://api.example.com
```

Pourquoi ces valeurs ?

- APP_PORT=3000 : Port standard de production
- NODE_ENV=production : Active les optimisations de Node.js
- LOG_LEVEL=warn : En production, on ne garde que les avertissements et erreurs (performance)
- API_URL : Pointe vers la vraie API de production

4.4 Structure des fichiers

```
tp8-gitops/  
environments/  
  staging.env      <-- Config pour le staging  
  production.env  <-- Config pour la production  
app.js  
app.test.js  
package.json
```

****Note importante**** : Dans un vrai projet, les ****secrets**** (mots de passe, clés API) ne doivent JAMAIS être da

Etape 5 : Créer un pipeline de déploiement avec promotion d'environnements

5.1 Le concept de promotion

La **promotion**, c'est faire avancer une version d'un environnement à l'autre :

```
Code --> [Tests] --> Staging (automatique) --> Production (manuel)
```

Pourquoi ?

- On déploie d'abord en **staging** pour tester "en vrai" sans risque
- Si tout va bien en staging, on **promeut** vers la production
- Le déploiement en production est **manuel** : un humain doit approuver

5.2 Créer les scripts de déploiement

Avant de créer le pipeline, on a besoin de deux scripts :

scripts/deploy.sh - **Script de déploiement**

Ce script simule un déploiement et crée un manifeste :

```
#!/bin/bash
# Script de deploiement GitOps
# Usage: ./scripts/deploy.sh <environnement>

ENVIRONMENT=$1

if [ -z "$ENVIRONMENT" ]; then
    echo "ERREUR: Vous devez specifier un environnement (staging ou production)"
    exit 1
fi

ENV_FILE="environments/${ENVIRONMENT}.env"

if [ ! -f "$ENV_FILE" ]; then
    echo "ERREUR: Le fichier $ENV_FILE n'existe pas"
    exit 1
fi

echo "====="
echo "  DEPLOIEMENT GitOps"
echo "  Environnement: $ENVIRONMENT"
echo "====="

# Charger la configuration
echo ""
echo "[1/5] Chargement de la configuration depuis $ENV_FILE..."
source "$ENV_FILE"
echo "  - APP_PORT=$APP_PORT"
echo "  - NODE_ENV=$NODE_ENV"
echo "  - LOG_LEVEL=$LOG_LEVEL"
echo "  - API_URL=$API_URL"

# Simuler les etapes de deploiement
echo ""
echo "[2/5] Installation des dependances..."
sleep 1
echo "  OK - Dependances installees"

echo ""
echo "[3/5] Construction de l'application..."
sleep 1
echo "  OK - Application construite"

echo ""
echo "[4/5] Deploiement vers $ENVIRONMENT..."
sleep 1
echo "  OK - Application deployee sur le port $APP_PORT"

# Creer le manifeste de deploiement
echo ""
echo "[5/5] Creation du manifeste de deploiement..."

VERSION="${APP_VERSION:-1.0.0}"
COMMIT_SHA="${CI_COMMIT_SHA:-$(git rev-parse HEAD 2>/dev/null || echo 'local')}"
TIMESTAMP=$(date -u +"%Y-%m-%dT%H:%M:%SZ")
DEPLOYER="${GITLAB_USER_LOGIN:-local-user}"

cat > deployment-manifest.json << MANIFEST_EOF
```

scripts/verify.sh - **Script de verification**

```
#!/bin/bash
# Script de verification de deploiement
# Usage: ./scripts/verify.sh

MANIFEST="deployment-manifest.json"

echo "=====
echo "  VERIFICATION DU DEPLOIEMENT"
echo "=====

# Verifier que le manifeste existe
if [ ! -f "$MANIFEST" ]; then
    echo "ERREUR: Le manifeste $MANIFEST n'existe pas"
    echo "Le deploiement n'a peut-etre pas ete effectue"
    exit 1
fi

echo ""
echo "[1/3] Lecture du manifeste de deploiement..."
echo "Contenu du manifeste:"
cat "$MANIFEST"

# Verifier les champs requis
echo ""
echo "[2/3] Verification des champs requis..."

REQUIRED_FIELDS=("version" "commit_sha" "timestamp" "environment" "deployer")
ALL_OK=true

for field in "${REQUIRED_FIELDS[@]"; do
    if grep -q "\"$field\"" "$MANIFEST"; then
        echo "  OK - Champ '$field' present"
    else
        echo "  ERREUR - Champ '$field' manquant!"
        ALL_OK=false
    fi
done

# Simuler un health check
echo ""
echo "[3/3] Simulation du health check..."
sleep 1

ENVIRONMENT=$(grep -o '"environment": "[^"]*"' "$MANIFEST" | cut -d'"' -f4)
VERSION=$(grep -o '"version": "[^"]*"' "$MANIFEST" | cut -d'"' -f4)

echo "  - Environnement: $ENVIRONMENT"
echo "  - Version: $VERSION"
echo "  - Statut: HEALTHY"

echo ""
echo "=====
if [ "$ALL_OK" = true ]; then
    echo "  VERIFICATION REUSSIE"
    echo "  Le deploiement est valide."
else
    echo "  VERIFICATION ECHOUUE"
    echo "  Des champs sont manquants dans le manifeste."

```

5.3 Rendre les scripts executables

```
chmod +x scripts/deploy.sh
chmod +x scripts/verify.sh
```

5.4 Creer le fichier .gitlab-ci.yml

Voici le pipeline complet. Lisez les commentaires pour comprendre chaque partie :

```
# =====
# Pipeline GitOps - TP8
# =====
# Ce pipeline implemente les principes GitOps:
# - Configuration declarative dans Git
# - Promotion d'environnements (staging -> production)
# - Manifeste de deploiement pour la tracabilite
# - Verification automatique des deploiements

variables:
  APP_VERSION: '1.0.0'
  NODE_VERSION: '18'
  DEPLOY_ENV: 'staging'

stages:
  - install
  - test
  - build
  - deploy-staging
  - deploy-production
  - verify

# --- Cache global pour les dependances ---
# Pourquoi un cache ? Pour ne pas retecharger node_modules a chaque job
cache: &global_cache
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
  policy: pull

# =====
# STAGE 1 : Installation des dependances
# =====
install-deps:
  stage: install
  image: 'node:${NODE_VERSION}'
  script:
    - 'echo "Installation des dependances..."'
    - npm ci
    - 'echo "Dependances installees avec succes"'
  cache:
    <<: *global_cache
    policy: pull-push

# =====
# STAGE 2 : Tests unitaires
# =====
test-unit:
  stage: test
  image: 'node:${NODE_VERSION}'
  script:
    - 'echo "Lancement des tests unitaires..."'
    - npm test
    - 'echo "Tests termines avec succes"'
  cache:
    <<: *global_cache

# =====
```

5.5 Comprendre le pipeline

```
install-deps --> test-unit --> build-app --> deploy-staging --> deploy-production --> verify-deployment
                                         (auto)                      (manuel)              (auto)
```

Points importants :

- deploy-staging se declenche **automatiquement** apres le build
- deploy-production a when: manual : il faut cliquer sur le bouton "Play" dans GitLab
- verify-deployment verifie que le deploiement s'est bien passe
- Les **artifacts** permettent de passer des fichiers entre les jobs

Etape 6 : Mecanisme de rollback avec Git

6.1 Le concept de rollback en GitOps

En GitOps, le rollback est simple : **on revient a un etat precedent dans Git.**

Pas besoin de scripts compliques. Git a deja tout l'historique !

```
Commit A (v1.0) --> Commit B (v1.1) --> Commit C (v1.2, bug!)
                                   |
                                   git revert C
                                   |
                                   Commit D (annule C, retour a v1.1)
```

6.2 Comment faire un rollback ?

Methode 1 : git revert (recommandee)

```
# Voir l'historique pour trouver le commit problematique
git log --oneline

# Annuler le dernier commit (cree un nouveau commit qui annule les changements)
git revert HEAD --no-edit

# Pousser le revert -> le pipeline se declenche automatiquement
git push
```

Pourquoi git revert et pas git reset ?

- git revert **creé un nouveau commit** qui annule les changements. L'historique est preserve.
- git reset **efface l'historique**. On perd la trace de ce qui s'est passe.
- En GitOps, on veut toujours garder l'historique complet (tracabilite).

Methode 2 : Redeployer une version specifique

```
# Voir les tags de version
git tag -l

# Se positionner sur une version specifique
git checkout v1.0.0

# Creer une branche de hotfix
git checkout -b hotfix/rollback-to-v1.0.0

# Pousser et creer une Merge Request
git push -u origin hotfix/rollback-to-v1.0.0
```

6.3 Le manifeste de deployment comme outil de rollback

Le fichier `deployment-manifest.json` cree a chaque deployment contient :

```
{
  "version": "1.0.0",
  "commit_sha": "abc123def456",
  "timestamp": "2026-04-06T10:30:00Z",
  "environment": "staging",
  "deployer": "etudiant1",
  "config": {
    "app_port": "3001",
    "node_env": "staging",
    "log_level": "debug",
    "api_url": "https://staging-api.example.com"
  }
}
```

Ce manifeste permet de savoir **exactement** ce qui est deploye a tout moment. Si on a un probleme, on sait quel commit a ete deploye et on peut revenir dessus.

Etape 7 : Variables specifiques par environnement

7.1 Comment ca fonctionne ?

Dans notre approche GitOps, chaque environnement a son propre fichier `.env` :

```
environments/
  staging.env      --> Variables pour staging
  production.env   --> Variables pour production
```

Le script `deploy.sh` charge le bon fichier selon l'environnement cible.

7.2 Ajouter des variables dans GitLab CI/CD

Pour les **secrets** (qui ne doivent pas etre dans Git), utilisez les variables GitLab :

1. Allez dans **Settings > CI/CD > Variables**
2. Cliquez sur **Add variable**
3. Ajoutez par exemple :
 - `DATABASE_URL` avec la valeur `postgresql://...`

- Cochez **Masked** pour cacher la valeur dans les logs
- Cochez **Protected** pour limiter aux branches protegees
- Dans **Environment scope**, choisissez staging ou production

7.3 Bonnes pratiques

A mettre dans Git	A mettre dans GitLab CI/CD
Ports, niveaux de log	Mots de passe
URLs non-sensibles	Cles API
Noms d'environnement	Tokens d'accès
Feature flags	Chaines de connexion BDD

Etape 8 : Manifeste de deployment et suivi des versions

8.1 Qu'est-ce qu'un manifeste de deployment ?

C'est un fichier JSON qui documente **chaque deployment**. Il répond aux questions :

- **Quoi ?** Quelle version a été déployée ?
- **Où ?** Sur quel environnement ?
- **Quand ?** A quelle date et heure ?
- **Qui ?** Qui a déclenché le deployment ?
- **Comment ?** Avec quelle configuration ?

8.2 Structure du manifeste

```
{
  "version": "1.0.0",
  "commit_sha": "le hash complet du commit",
  "timestamp": "date et heure UTC",
  "environment": "staging ou production",
  "deployer": "nom de l'utilisateur GitLab",
  "config": {
    "app_port": "3001",
    "node_env": "staging",
    "log_level": "debug",
    "api_url": "https://staging-api.example.com"
  }
}
```

8.3 Pourquoi c'est important ?

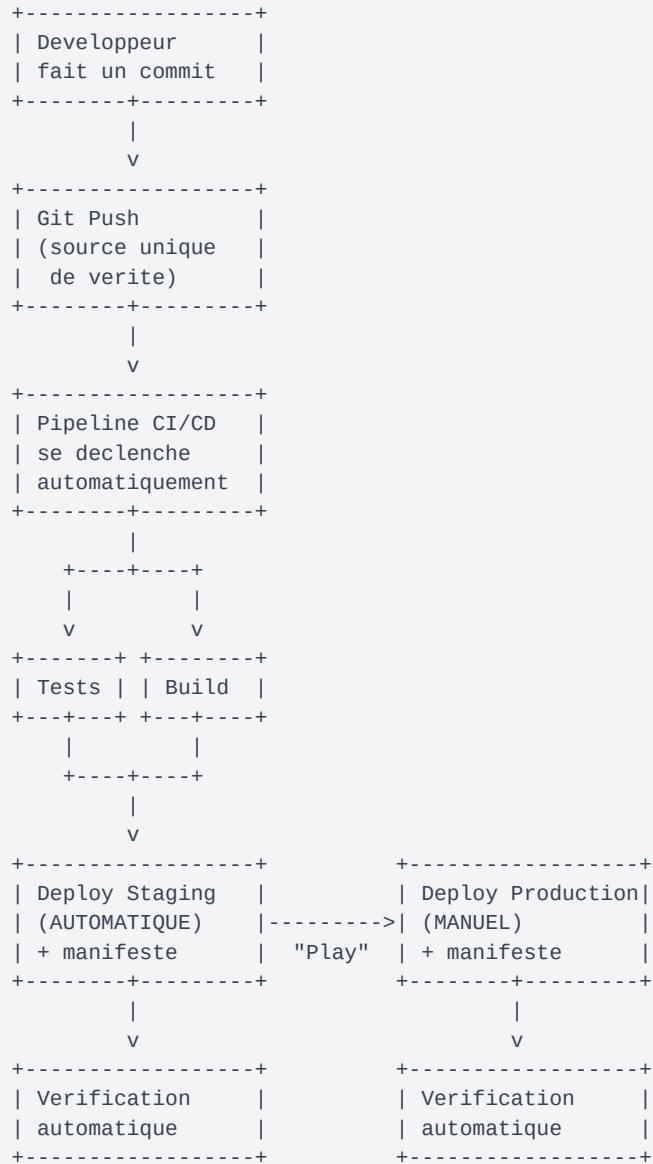
Imaginez ce scénario :

Il est 3h du matin. L'application de production est en panne. Le manifeste de deployment vous dit immédiatement

Sans le manifeste, vous passeriez des heures à chercher ce qui a changé.

Etape 9 : Le workflow GitOps complet

9.1 Diagramme du flux complet



9.2 Mettre tout ensemble

Voici l'ordre des commandes pour lancer le TP complet :

```
# 1. Cloner le projet
git clone https://gitlab.com/VOTRE-USERNAME/tp8-gitops.git
cd tp8-gitops

# 2. Verifier que tous les fichiers sont presents
ls -la
ls -la environments/
ls -la scripts/

# 3. Ajouter tous les fichiers
git add .

# 4. Faire le premier commit
git commit -m "feat: mise en place du pipeline GitOps"

# 5. Pousser vers GitLab
git push origin main

# 6. Aller sur GitLab > CI/CD > Pipelines pour voir le resultat

# 7. Une fois le staging deploye, cliquer sur "Play" pour la production
```

9.3 Tester le rollback

```
# Faire un changement (simuler un "bug")
echo "// Bug introduit" >> app.js
git add app.js
git commit -m "feat: ajout d'une fonctionnalite (avec bug)"
git push

# Attendre que le pipeline se termine...

# Faire le rollback avec git revert
git revert HEAD --no-edit
git push

# Le pipeline se declenche a nouveau avec la version corrgee !
```

Tableau recapitulatif

Concept	Explication	Ou dans le TP
Source unique de verite	Tout est dans Git	Tout le depot
Configuration declarative	Fichiers .env par environnement	environments/
Promotion	staging -> production	Pipeline CI/CD
Deploiement automatique	Staging se deploye a chaque push	deploy-staging
Deploiement manuel	Production necessite approbation	deploy-production (when: manual)
Manifeste de deploiement	Trace de chaque deploiement	deployment-manifest.json
Rollback	Retour a une version precedente	git revert
Verification	Controle post-deploiement	verify-deployment
Secrets	Variables sensibles hors de Git	Variables GitLab CI/CD

Erreurs courantes

1. "Le pipeline ne se declenche pas"

Cause probable : Vous n'etes pas sur la bonne branche.

Solution : Verifiez que vous poussez sur main ou master (le pipeline a only: main, master).

```
git branch # Verifier la branche actuelle
git push origin main
```

2. "Permission denied sur les scripts"

Cause probable : Les scripts ne sont pas executables.

Solution :

```
chmod +x scripts/deploy.sh
chmod +x scripts/verify.sh
git add scripts/
git commit -m "fix: rendre les scripts executables"
git push
```

3. "Le fichier .env n'existe pas"

Cause probable : Le dossier environments/ n'a pas ete commite.

Solution :

```
git add environments/
git commit -m "fix: ajouter les fichiers de configuration"
git push
```

4. "Le manifeste est vide ou incomplet"

Cause probable : Les variables d'environnement ne sont pas chargees.

Solution : Verifiez que le fichier .env est bien au format CLE=va leur sans espaces autour du =.

5. "Le deploy en production ne se lance pas"

Cause probable : C'est normal ! Le deploiement en production est **manuel**.

Solution : Allez dans GitLab > CI/CD > Pipelines > cliquez sur le bouton "Play" (triangle) a cote du job deploy-production.

6. "Le job verify-deployment echoue"

Cause probable : Le manifeste de deploiement n'a pas ete passe entre les jobs.

Solution : Verifiez que deploy-staging cree bien un artifact deployment-manifest.json et que verify-deployment a dependencies: deploy-staging.

Pour aller plus loin

- **ArgoCD** : Outil GitOps pour Kubernetes (modele Pull)
- **Flux** : Autre outil GitOps pour Kubernetes
- **Kustomize** : Gestion de configuration Kubernetes par environnement
- **Helm** : Gestionnaire de paquets Kubernetes

Bravo ! Vous avez mis en place un workflow GitOps complet. Le principe fondamental a retenir :

****Si ce n'est pas dans Git, ca n'existe pas.****

Tout changement doit passer par Git. Tout deploiement doit etre tracable. Tout doit pouvoir etre reproduit.